

LOCAL LLM

OLLAMA

STREAMLIT

PYTHON

WEATHER FIT *talk.*

날씨 × 패션 AI 스타일링 챗봇

PROJECT

WEATHER FIT TALK

SPEAKER

박수용

STACK

Python · Ollama · Streamlit

MODEL

Gemma · 로컬 실행

오늘 함께 살펴볼 네 가지 이야기.

CH · 01

INTRO

프로젝트 소개

- 한 줄 소개
- 왜 만들었나
- 핵심 기능 8가지

P. 03 — 05

CH · 02

STRUCTURE

구조와 흐름

- 사용 기술 스택
- 전체 동작 흐름
- 프로젝트 파일 구조

P. 06 — 08

CH · 03

CODE

핵심 파일 3개

- app.py
- prompts.py
- tools.py

P. 09 — 11

CH · 04

CLOSING

실행 · 마무리

- 통신 방식 (Python → Ollama)
- 실행 방법
- 장점 & 향후 계획

P. 12 — 15

오늘 날씨와 상황에 맞는 코디를 AI가 추천해주는 챗봇.

USER

"오늘 비 오고 바람이 불어. 학교 갈 때 뭐 입을까?"

WEATHER FIT TALK

방수 바람막이 + 어두운 데님 + 미끄럼방지 스니커즈.
작은 우산까지 챙기시면 좋을 것 같아요.

RECOMMENDS

상의

하의

아우터

신발

소품

매일 아침의 작은 고민에서 시작되었습니다.

PROBLEM 01

"오늘 뭐 입지?"

매일 날씨에 맞춰 옷을 고르는 일은 생각보다 손이 많이 가는 결정입니다.

PROBLEM 02

날씨 앱은 옷차림을 알려주지 않는다

기온과 강수확률은 있지만, 구체적인 코디까지 제안해 주지는 않습니다.

IDEA 03

시라면 날씨 · 상황 · 스타일을 한 번에 고려할 수 있다

조건을 종합해 사람이 골라주는 듯한 추천을 만들고 싶었습니다.

IDEA 04

직접 만들어보는 로컬 LLM 경험

내 컴퓨터에서 직접 돌아가는 AI(로컬 LLM)의 구조를 이해하며 만들어보고 싶었습니다.

사용자의 한마디에서 8가지 기능이 작동합니다.

F · 01

날씨 기반 코디 추천

기온·바람·강수 조건에 맞춰 옷차림을
자동 제안

F · 02

상황별 추천

등교 · 발표 · 면접 · 데이트 등 TPO 반
영

F · 03

스타일 선택

미니멀 · 캐주얼 · 스트릿 · 포멀 중 선택

F · 04

사용자 입력 우선 반영

사이드바 조건보다 직접 쓴 문장을 먼
저 적용

F · 05

Ollama 로컬 LLM 답변

내 PC에서 직접 돌아가는 AI 모델로
답변 생성

F · 06

DuckDuckGo 검색 활용

선택 시 실시간 검색 결과로 답변 풍부
화

F · 07

오프라인 실행 가능

인터넷 연결 없이 내 PC에서 바로 동
작

F · 08

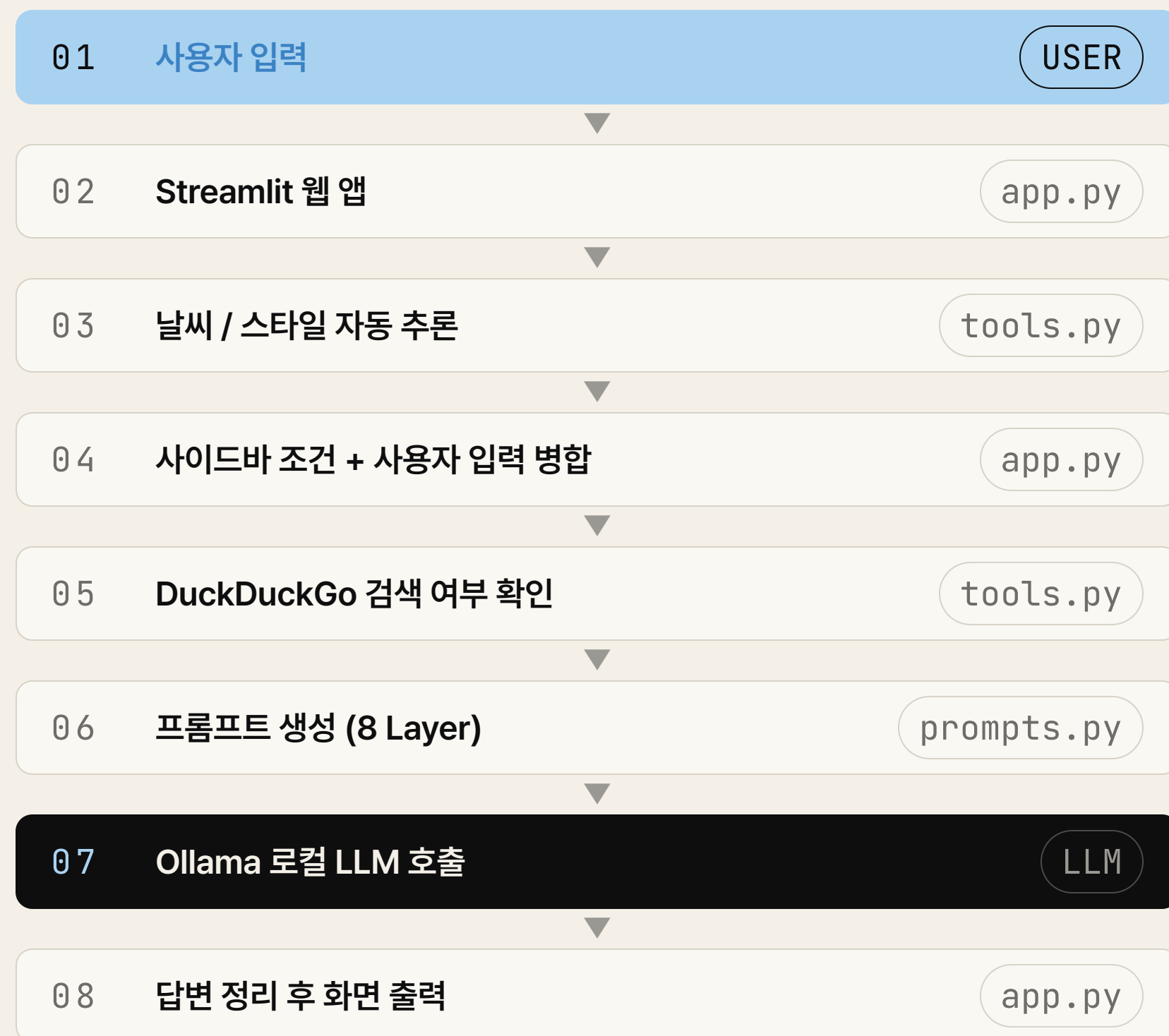
Streamlit 웹 UI

설치 후 브라우저에서 바로 사용 가능

무료 · 오픈소스로만 구성된 가벼운 스택.

#	분 류	기 술	한 줄 설 명
01	언어	Python	전체 로직을 짤 프로그래밍 언어 (초보자에게 가장 친숙한 언어)
02	웹 프레임워크	Streamlit	파이썬 코드만으로 웹 화면을 만들 수 있는 도구
03	LLM 실행 환경	Ollama	내 컴퓨터에서 AI 모델을 직접 돌릴 수 있게 해주는 프로그램
04	사용 모델	Gemma 4:e4b	구글이 공개한 오픈소스 AI 모델 (실제로 답변을 만들어 주는 두뇌)
05	웹 검색	DuckDuckGo (ddgs)	필요할 때 인터넷 검색 결과를 가져와 AI 답변에 활용
06	통신	requests	Ollama 서버에 질문을 보내고 답을 받아오는 통신 도구
07	스타일링	CSS	웹 화면의 색상, 카드, 버튼 등 디자인을 담당

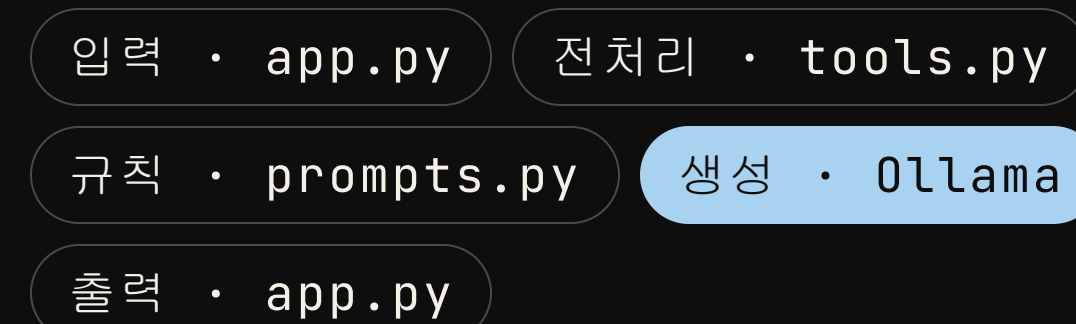
질문 한 줄 → 답변까지, 여덟 걸음의 여정.



흐름의 핵심 한 줄

사용자의 한 문장이
tools.py에서 정리되고,
prompts.py의 규칙에 실려
Ollama LLM으로 전달됩니다.

단계별 책임



파일은 단 6개, 핵심은 그중 3개입니다.

```
Weather-Fit-Talk-Local-LLM/
```

```
|—— app.py # 메인 실행
```

```
|—— prompts.py # AI 규칙
```

```
|—— tools.py # 보조 도구
```

```
|—— style.css # 디자인
```

```
|—— requirements.txt # 라이브러리
```

```
|—— README.md # 설명서
```

app.py	웹 화면, 사용자 입력, 사이드바, Ollama 호출을 모두 담당하는 메인 파일
prompts.py	AI가 어떤 말투·형식으로 답할지 지시하는 프롬프트 규칙
tools.py	날씨/스타일 추론과 검색 결과를 정리해 주는 보조 도구 함수
style.css	웹 화면의 색상, 카드, 버튼 등 디자인 (CSS = 스타일 시트)
requirements.txt	설치해야 할 파이썬 라이브러리 목록
README.md	프로젝트가 무엇이고 어떻게 실행하는지 적은 설명 문서

사용자와 AI가 만나는 메인 화면.

ROLE · 역할

app.py ·

사용자와 AI를 잇는 메인 파일

사용자의 입력을 받고, 필요한 정보를 모은 뒤
AI에게 요청을 보내고,
답변을 다시 화면에 정리해 보여줍니다.

INPUT → 처리 → AI 호출 → OUTPUT

01 입력 · 지역 / 날씨 / 기온 / 상황 / 스타일 선택

↓

02 입력 · 채팅창에 질문 한 줄 작성

↓

03 처리 · 입력값을 모아 프롬프트 생성

↓

04 AI 호출 · Ollama 로컬 LLM에 전달

↓

05 출력 · 답변을 화면에 정리해서 표시

AI에게 답변 규칙을 전달하는 지침서.

ROLE · 역할

prompts.py ·

AI 답변 규칙 모음

AI가 어떤 말투로 답할지,
어떤 형식으로 추천할지,
그리고 하면 안 되는 답변이 무엇인지를
미리 정의해 둡니다.

8개의 레이어로 단계별 지시.

RESULT

매번 비슷한 톤으로,
같은 형식의 코디 답변이 나옵니다.

8 LAYERS · 프롬프트 구성

SYSTEM_LAYER

AI의 역할을 정의 (스타일
링 챗봇)

SAFETY_LAYER

HTML / CSS 코드
출력 금지

USER_INPUT_PRIORITY

사용자가 쓴 문장을 가장 우선
시

STYLE_PRIORITY

선택한 스타일을 충실
히 반영

WEB_SEARCH_LAYER

검색 결과를 어떻게 활용할
지 규정

RULE_LAYER

답변 작성 시 지켜야
할 규칙

RECOMMENDATION

코디 추천의 기준 (상의·하의·신
발 등)

OUTPUT_FORMAT

최종 답변의 형식·순서
를 고정

AI 답변을 더 똑똑하게 만드는 보조 함수 모음.

ROLE · 역할

tools.py ·

전처리 · 검색 · 요약 함수

날씨 조건을 정리하고,
어울리는 스타일을 판단하고,
검색 결과를 짧게 요약해서
AI가 더 정확한 답변을 만들도록 돕습니다.

CORE IDEA

"정제된 입력 → 더 정확한 답변"

사용자 입력 + 사이드바 조건을 깔끔하게 한 덩어리로 정리합니다.

TOOLS · 6 FUNCTIONS

T · 01

날씨 조건 추론

기온·바람·강수 조합을
한 줄로 요약

T · 02

스타일 자동 추천

상황 키워드로 어울리는
스타일 판단

T · 03

검색 키워드 생성

질문에서 검색에 쓸 단어
를 뽑아냄

T · 04

DuckDuckGo 검색

필요할 때만 실시간 검색
을 실행

T · 05

검색 결과 요약

긴 결과를 짧게 정리해
AI에 전달

T · 06

입력 병합 보조

사용자 입력 + 사이드바
를 합쳐 정돈

app.py → tools.py → prompts.py → [Ollama](#)

정리된 입력이 LLM까지 한 흐름으로 전달됩니다

Python이 AI에게 요청을 보내는 법.



REQUEST 보내는 데이터

model	: "gemma4:e4b" 사용 모델 지정
prompt	: prompts.py에서 만든 전체 지시문
stream	: False 한 번에 완성된 답변 받기
options	: temperature 0.4 일관된 어조

RESPONSE 받는 데이터

response	: AI가 생성한 코디 추천 본문
model	: 실제 사용된 모델 이름
done	: 생성 완료 여부 (true/false)
총 흐름	: 요청 → 대기 → 응답 → 화면 출력

STRUCTURE 프론트 / 백 구조

Frontend	: Streamlit · 웹 UI, 입력, 화면 출력
Backend	: Ollama 로컬 서버 · 모델 실행, 답변 생성
연결 방식	: Python requests 라이브러리 HTTP 통신

REALTIME 실시간 통신 여부

방식	: 단방향 요청-응답 (WebSocket 미사용)
stream	: False · 답변 완성 후 한 번에 전송
이유	: 안정성 우선, 부분 출력보다 완성된 답변이 적합

대화를 기억하는 `st.session_state`의 역할.

ROLE · 역할

`st.session_state`

대화 기록을 유지하는 저장소

Streamlit은 사용자가 입력할 때마다 페이지가 새로 고쳐지는 구조입니다. 이때 이전 대화가 사라지지 않도록 `st.session_state`에 메시지를 저장합니다.

01 저장
사용자가 입력하면 → `st.session_state` 리스트에 `role` 과 `content` 를 딕셔너리로 추가

02 유지
페이지가 새로고침되어도 → `session_state` 는 초기화되지 않고 대화 기록이 유지됨

03 전달
AI 호출 시 → 최근 `6개` 메시지를 꺼내 대화 맥락으로 프롬프트에 포함

CORE IDEA

“새로고침해도 대화가 유지되는 이유”

`st.session_state`가 브라우저 세션 동안 메모리를 보존하기 때문입니다.

대화 초기화 버튼을 누르면 `session_state`를 비워 처음 상태로 돌아갑니다.

설계부터 개선까지, 6단계로 만들었다.

STEP 01	프로젝트 구조 설계 파일 역할 분리 (app / prompts / tools)
STEP 02	tools.py 구현 날씨 키워드 감지, 스타일 추론, 병합 함수 작성
STEP 03	prompts.py 구현 8개 레이어 프롬프트 설계 및 출력 포맷 정의
STEP 04	app.py 구현 Streamlit UI, 사이드바, Ollama 호출, 화면 출력 연결
STEP 05	CSS 디자인 적용 블랙 × 아이보리 에디토리얼 스타일 구현
STEP 06	테스트 및 프롬프트 튜닝 날씨 조건별 답변 품질 확인, 레이어 규칙 수정

CORE LOOP

설계 → 도구 → 규칙 → 연결 → 디자인 →
개선

작게 나누고, 하나씩 붙이며,
마지막에 다듬는 6단계의 흐름.

개발 중 만난 문제, 코드로 해결한 과정.

PROBLEM 01

01

AI 답변에 HTML 코드가 섞여 출력됨

로컬 LLM이 스타일 태그를 함께 생성하는 문제

SOLUTION

- `clean_text()` 함수 구현
- 정규표현식으로 HTML/CSS 태그를 전부 제거
- 답변에서 순수 텍스트만 추출해 화면에 출력

PROBLEM 02

02

사이드바 설정과 사용자 말이 충돌

“맑음” 선택 상태에서 “오늘 비 와”라고 입력 시 혼동 발생

SOLUTION

- 사용자 입력 우선 원칙 설계
- `infer_weather_from_user_input()` 으로 문장에서 날씨 감지
- `merge_user_weather_with_sidebar()` 로 사용자 말을 우선 적용

문제를 만나면 원인을 찾고, 코드로 해결한다.

8단계만 따라하면 지금 바로 실행 가능.

- 01 프로젝트 클론
- 02 가상환경 생성
- 03 라이브러리 설치
- 04 Ollama 설치 확인
- 05 모델 다운로드
- 06 Ollama 모델 실행
- 07 Streamlit 실행
- 08 localhost:8501 접속

RUN

```
terminal · WEATHER-FIT-TALK

# 1. 프로젝트 받기
$ git clone .../Weather-Fit-Talk-Local-LLM.git

# 2. 가상환경
$ python -m venv .venv
$ .\.venv\Scripts\activate

# 3. 라이브러리 설치
$ pip install -r requirements.txt

# 4. Ollama 모델 받기 + 실행
$ ollama pull gemma3:4b
$ ollama run gemma3:4b

# 5. Streamlit 실행
$ python -m streamlit run app.py
✓ http://localhost:8501
```


단순한 챗봇이 아니라, 실용적인 로컬 AI 서비스로.

지금 잘하는 것 STRENGTHS

- 로컬 LLM 기반, 내 PC에서 직접 실행
- 인터넷 없이도 동작하는 오프라인 챗봇
- 코드 구조를 직접 이해할 수 있는 학습용 프로젝트
- 날씨와 패션을 결합한 실용적 서비스
- Streamlit으로 빠른 웹 UI 구현

느낀 점 WHAT I LEARNED · 배운 점

- 로컬 LLM을 직접 연결하며 AI가 단순한 API가 아님을 이해했다
- 프롬프트 설계가 코드만큼 중요하다는 걸 느꼈다
- 사용자 입력을 우선하는 로직 설계에서 UX 사고방식을 배웠다

앞으로의 계획 NEXT

- 실제 날씨 API 연동
- 지역 자동 감지
- 스타일 이미지 추천
- 사용자 옷장 데이터 저장
- 모바일 반응형 / 배포 / 저장 기능

WEATHER FIT TALK는 단순한 챗봇이 아니라,
날씨와 상황을 이해해 사용자에게 **실용적인 스타일링**을
제안하는 로컬 AI 서비스입니다.

감사합니다

오늘 발표를 들어주신 모든 분께 감사드립니다.
궁금한 점은 편하게 질문해 주세요.

Q & A

자유로운 질문 환영합니다

PROJECT

WEATHER FIT TALK

SPEAKER

박수용